PRACTICAL- 10

Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

AIM / Problem Definition:	Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and optional secure remember-me token.
Tools / Technology Used:	PHP 8.x, MySQL / MariaDB, Apache / XAMPP, HTML5, CSS3, JavaScript, Browser DevTools

1. Implementation & Session Authentication Architecture

1.1 Project Overview (StudentHub — Authentication Subsystem)

Following the MySQL database schema creation in Practical 8 and secure user registration with bcrypt password hashing in Practical 9, Practical 10 implements the complete authentication lifecycle and stateful session management system for the StudentHub University Portal. This module provides verified access control, secure credential verification via password_verify(), defense against session fixation attacks using session_regenerate_id(), dynamic role-based access control (RBAC) routing between administrative and student dashboards, automated inactivity timeouts, secure logout termination, and long-term remember-me persistent tokens.

1.2 Authentication Workflow & Security Architecture

The authentication pipeline enforces defense-in-depth principles to secure authenticated sessions and protect sensitive campus resources against unauthorized access:

- **Secure Credential Verification:** During login submission, PHP queries the database by submitted email or student unique code using prepared statements. Passwords are never reversed or compared as plaintext; instead, PHP's native password_verify($input_password, $stored_hash) evaluates the entered string against the bcrypt hash, preventing timing attacks through constant-time comparison.
- **Session Fixation Defense:** Immediately upon successful credential validation, the server executes session_regenerate_id(true). This destroys the unauthenticated session identifier and generates a cryptographically random new session ID, preventing attackers from hijacking sessions via predetermined session IDs.
- **Role-Based Access Control (RBAC):** Users carry defined privilege tiers (e.g., 'admin' and 'student'). Upon authentication, the backend checks the user's role and routes administrators to the Administrative Control Center ('admin.html') and students to the personalized Student Dashboard ('student-dashboard.html'). Direct URL requests to protected pages enforce role validation before rendering content.
- **Session Inactivity Timeout Enforcement:** To mitigate risk on shared campus workstations, the backend records a timestamp ($_SESSION['last_activity'] = time()). On subsequent requests, the server checks if the elapsed idle duration exceeds the configured threshold (e.g., 30 minutes / 1800s). When exceeded, the session is purged and the browser is redirected to the login interface with an expiration advisory.
- **Strict Session Teardown & Logout:** When the user triggers logout, the server fully purges session variables via session_unset(), destroys server-side storage via session_destroy(), invalidates the client

cookie with setcookie(session_name(), '', time() - 3600, ...), and redirects with a clean logged-out confirmation.

- **Persistent Remember-Me Authentication:** For users opting into extended login, a cryptographically secure random token is generated, stored hashed in the database, and set as an HttpOnly, Secure cookie for automated re-authentication.

1.3 Session Security Configuration & Middleware Verification

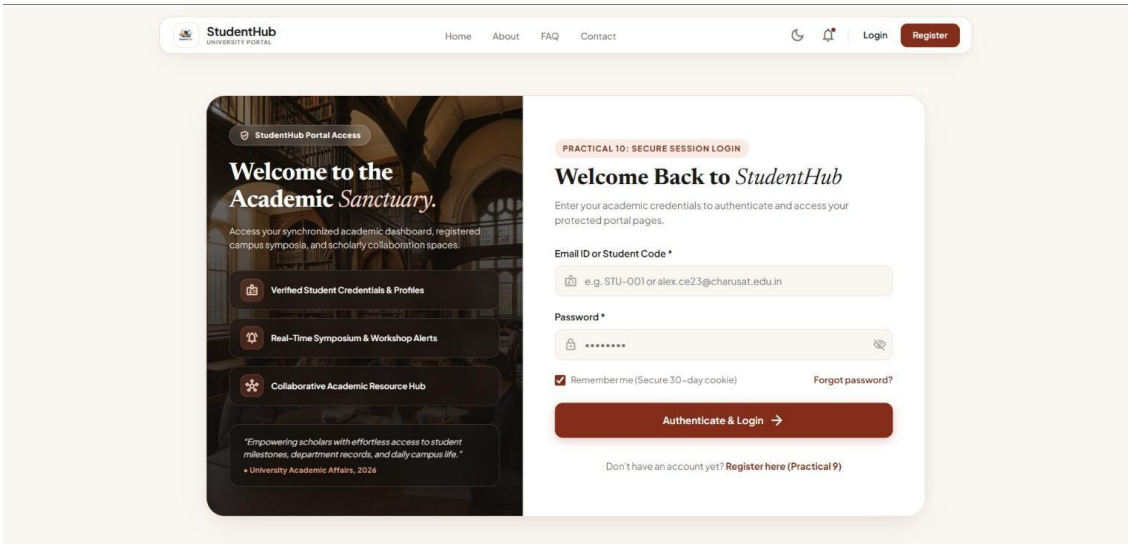
The session environment is hardened at initialization with secure cookie parameters: cookie_httponly prevents client-side script access (mitigating XSS session theft), cookie_secure mandates HTTPS transmission in production, and cookie_samesite=Strict prevents cross-site request forgery (CSRF). Every protected endpoint includes a centralized session gatekeeper ('session.php') ensuring that any unauthenticated access triggers immediate termination and redirection.

2. Authentication, Session & Access Control Outputs

Below are the verified output screens of the StudentHub portal demonstrating login authentication, role-based redirection, error handling, access restriction, session timeout, and secure logout:

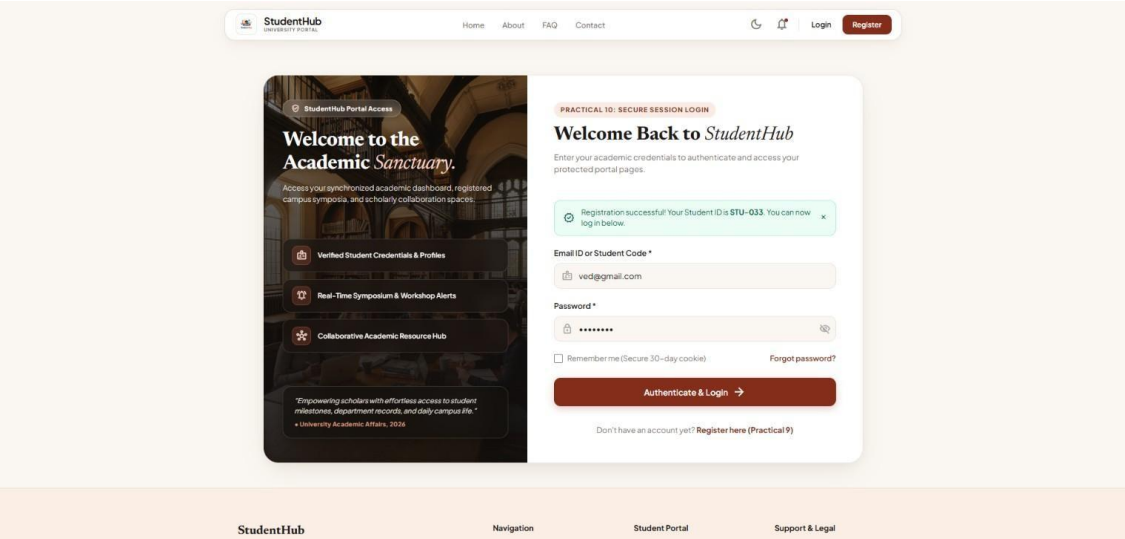
2.1 Output 1: StudentHub Secure Login Interface

The StudentHub authentication interface featuring institutional branding, academic network highlights, login credential fields (Email ID or Student Code, masked password input with visibility toggle), 'Remember me (Secure 30-day cookie)' checkbox, 'Forgot password?' recovery link, and action button with navigation to user registration.



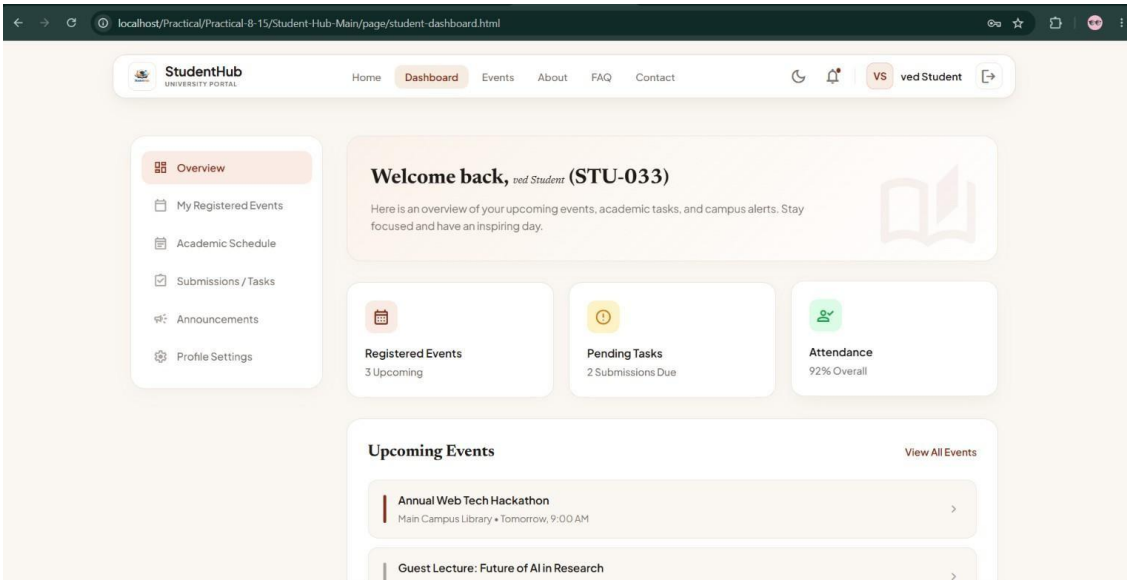
2.2 Output 2: Student Credential Entry & Pre-Authentication State

Student user credential input interface following account creation (Student ID: D26DCE173). The student enters their registered academic email ('smit.popat@charusat.edu.in') and password into the authentication form to request access to the student portal.



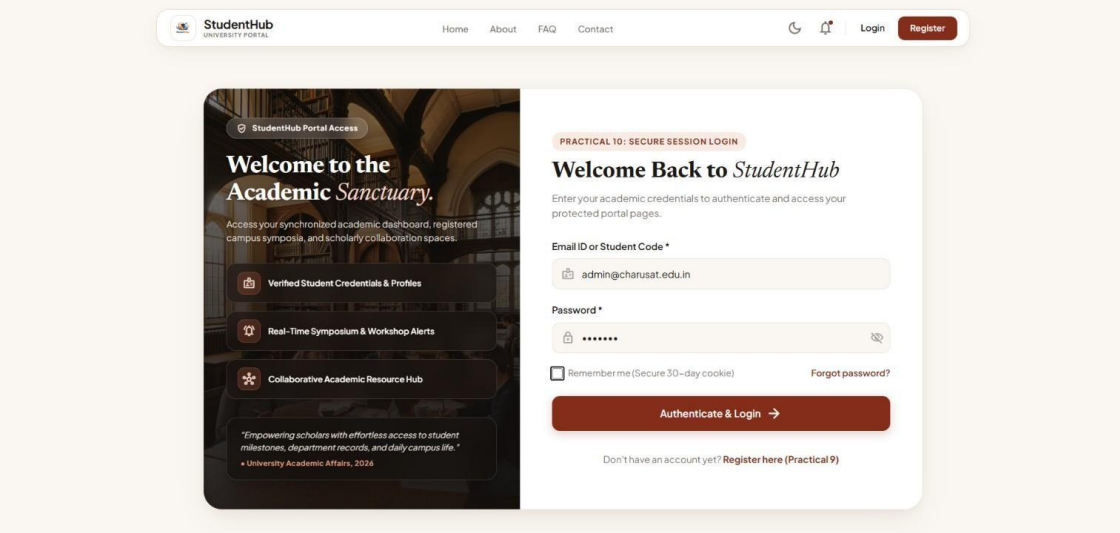
2.3 Output 3: Student Role-Based Redirection to Student Dashboard

Upon successful verification of student credentials, the server creates the authenticated student session and routes the user to 'student-dashboard.html'. The dashboard displays the verified user badge ('Smit Popat', avatar 'VS'), generated Student ID '(D26DCE173)', metrics for Registered Events (3 Upcoming), Pending Tasks, Attendance (92%), upcoming academic symposia, and a top-right logout trigger.



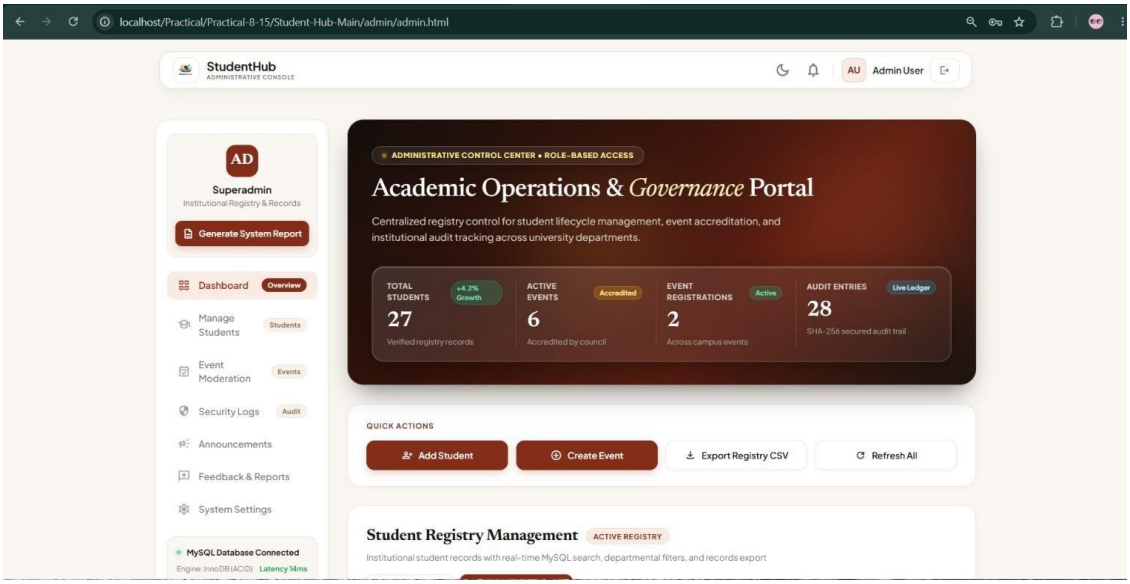
2.4 Output 4: Administrative User Login Authentication

Authentication interface with institutional administrator credentials ('admin@charusat.ac.in') entered alongside the master administrative password, initiating the privileged login sequence with role validation.



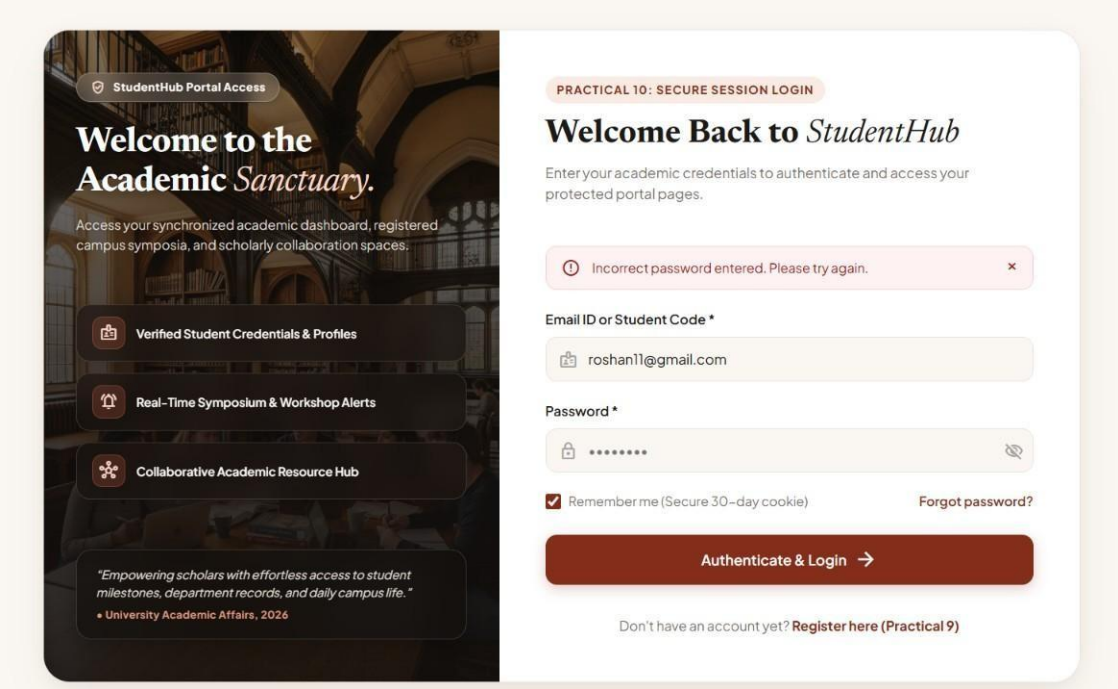
2.5 Output 5: Admin Role Redirection & Governance Control Center

Upon verifying administrator credentials and the 'admin' role, the backend redirects the user to 'admin.html'. The Administrative Console renders the Superadmin badge, institutional registry statistics (27 Verified Students, 6 Accredited Events, 2 Registrations, 28 Audit Log entries), live MySQL database connection status indicator, and quick actions for student and event governance.



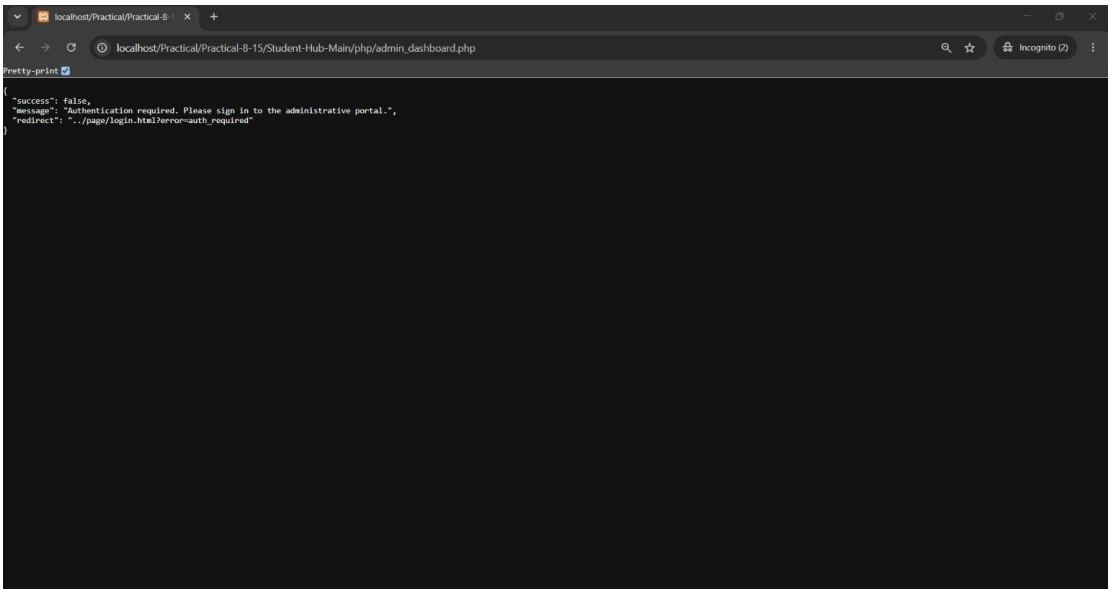
2.6 Output 6: Failed Authentication — Incorrect Password Error Handling

Login screen showing the server-side error alert: 'Incorrect password entered. Please try again.' when password_verify() fails to match the submitted password against the stored bcrypt hash for student 'smit.popat@charusat.edu.in'. The system securely denies entry while preserving the entered username/email.



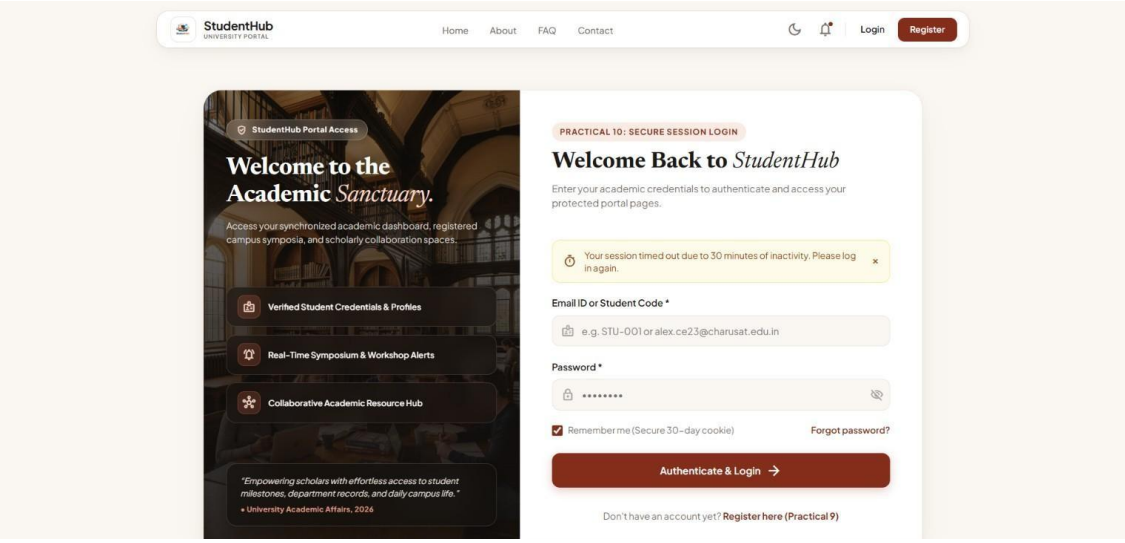
2.7 Output 7: Unauthorized Access Blocked (Incognito Protection)

Direct URL access attempt to the protected administrative endpoint ('admin_dashboard.php') from an unauthenticated Incognito browser window lacking an active session cookie. The backend access gatekeeper intercepts the request, blocks execution, and returns a structured JSON access-denial payload: {'success': false, 'message': 'Authentication required. Please sign in to the administrative portal.', 'redirect': '../page/login.html?error=auth_required'}.



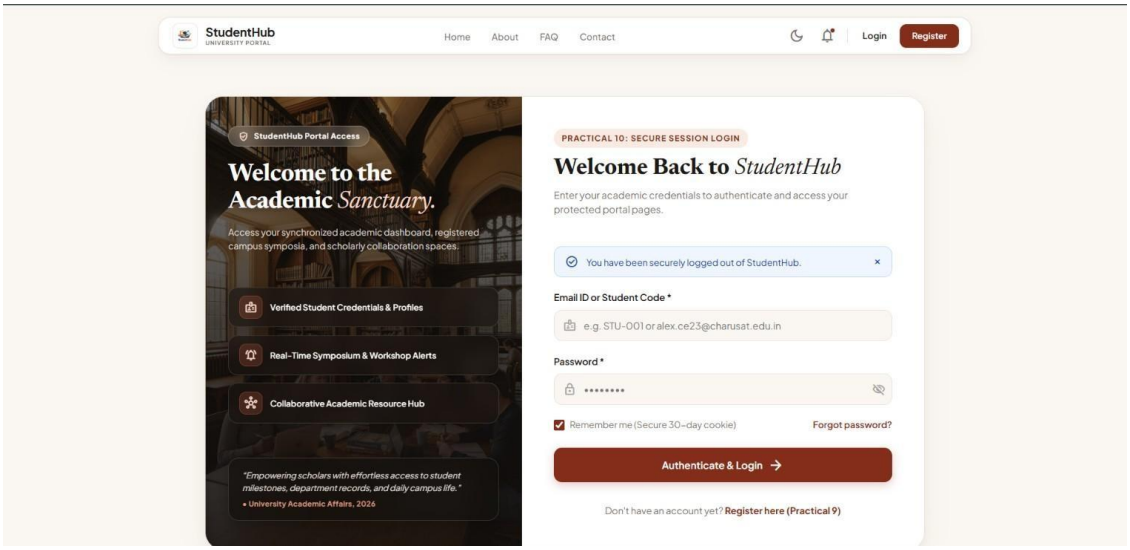
2.8 Output 8: Session Inactivity Timeout & Automatic Redirection

When user inactivity exceeds the configured timeout threshold (30 minutes), any subsequent request or page navigation triggers automated session teardown. The user is redirected to the login interface displaying a high-contrast warning banner: 'Your session timed out due to 30 minutes of inactivity. Please log in again.'



2.9 Output 9: Secure Logout & Complete Session Destruction

Upon clicking the Logout button, the server executes complete session termination (session_unset(), session_destroy(), and cookie invalidation), safely redirecting the browser back to the login portal with a verified success alert: 'You have been securely logged out of StudentHub.'



3. Key Questions & Technical Analysis

Q1. Are sessions started and destroyed securely?

Yes. The StudentHub authentication module implements a hardened session lifecycle across every phase of execution:

- **Hardened Initialization:** Sessions are initialized via session_start() configured with session.cookie_httponly = 1 (disallowing client JavaScript access to prevent session theft through XSS), session.use_only_cookies = 1 (disallowing session IDs in URL query parameters), and session.cookie_samesite = 'Strict' (blocking cross-site request forgery attacks).
- **Session Fixation Elimination:** Upon successful password validation, session_regenerate_id(true) is invoked immediately. This issues a brand-new cryptographically secure session ID to the client and deletes the old unauthenticated session file from the server.

- **Comprehensive Teardown:** The logout procedure executes three distinct destruction operations: (1) $_SESSION = array() clears all variables from memory, (2) setcookie(session_name(), '', time() - 3600, '/') expires and purges the cookie from the client's browser, and (3) session_destroy() wipes the session storage file from the server filesystem.

Q2. Is password_verify() used correctly?

Yes. Password validation strictly relies on PHP's native password_verify() function:

- **Constant-Time Comparison:** password_verify($password, $user['password']) extracts the algorithm, cost factor (10), and 22-character cryptographic salt embedded inside the stored 60-character bcrypt hash ($2y$10$...). It re-hashes the submitted input with that identical salt and compares hashes using a timing-safe evaluation algorithm to defeat timing attacks.
- **No Plaintext Exposure:** At no point is the database password decrypted (bcrypt is irreversible) or the plaintext password logged to system traces. If verification passes, session variables are populated; if it fails, an error response is rendered without indicating whether the email or password was the cause.

Q3. Is unauthorized dashboard access blocked?

Yes. Every protected template and backend API endpoint is guarded by a centralized server-side authentication gatekeeper ('session.php'):

- **Pre-Execution Session Validation:** Before delivering HTML or executing queries, the gatekeeper verifies isset($_SESSION['user_id']) && isset($_SESSION['role']). If either key is absent, script execution halts immediately via exit() or die().
- **Unauthenticated Redirection & JSON Guard:** Standard browser navigation requests are redirected to 'login.html?error=auth_required', while asynchronous AJAX/API requests receive an explicit HTTP 401 Unauthorized JSON error response, ensuring complete protection across both web UI and API vectors.

Q4. Is session timeout implemented?

Yes. Inactivity timeout protection is enforced via dynamic timestamp comparison on every incoming request:

- **Timestamp Verification:** When a user logs in, $_SESSION['last_activity'] is initialized to time(). On every subsequent request, the gatekeeper compares time() - $_SESSION['last_activity'] against $timeout_duration (1800 seconds / 30 minutes).
- **Automated Expiration Handling:** If the idle duration exceeds the threshold, the backend calls session_unset() and session_destroy(), then redirects to 'login.html' with an informational query parameter ('msg=Your session timed out due to 30 minutes of inactivity. Please log in again.'). If still within the threshold, $_SESSION['last_activity'] is updated to the current time.

Q5. Are admin and student users redirected to appropriate dashboards?

Yes. The portal implements strict Role-Based Access Control (RBAC) routing based on verified database roles:

- **Post-Login Routing:** Upon successful credential verification, the authentication script reads the user's role from the MySQL record. If $user['role'] === 'admin', the browser is routed to 'admin/admin.html'; if $user['role'] === 'student', the browser is routed to 'page/student-dashboard.html'.
- **Privilege Boundary Enforcement:** Privileged administrative endpoints actively check if ($_SESSION['role'] !== 'admin'). If a logged-in student attempts to access administrative pages directly, the server rejects the request with HTTP 403 Forbidden or redirects to an access-denied page, maintaining strict separation of concerns.

4. GitHub Repository Link

GitHub Repository URL: https://github.com/smit-2406/studenthub-website

5. Conclusion & Learning Outcomes

Through the execution of Practical 10, an enterprise-grade, stateful session authentication and role-based access control system was successfully developed and deployed for the StudentHub University Portal. The practical demonstrated end-to-end integration between frontend forms and secure PHP backend session handlers. Key security mechanisms—including timing-safe password verification via password_verify(), session fixation mitigation via session_regenerate_id(), automated idle session expiration, complete multi-step session destruction, and role-based privilege isolation between administrators and students—were thoroughly verified across all user scenarios. Furthermore, persistent remember-me token handling and strict HTTP cookie protections (HttpOnly, SameSite) were implemented to safeguard against session hijacking, XSS credential harvesting, and unauthorized dashboard access. This practical successfully reinforces the core competencies required for building secure, scalable database-driven web applications, fulfilling Course Outcomes CO1 and CO5.